集合通信 · NCCL · Overlap

04

大模型训练中的通信方式

> **原始问题：**详细讲解大模型训练中所有通信方式，重点拆开讲解，说明应用位置、时机以及能否和计算 overlap。

提问：2026/07/26 15:26　更新：2026/07/27 01:53　重要度：5/5

一句话结论

大模型训练中的通信只有两大类：集合通信(Collective)和点对点通信(P2P)。集合通信是“一群 GPU 一起参与”(AllReduce/AllGather/ReduceScatter/All-to-All/Broadcast)，由 NCCL 实现；点对点是“一张卡发给另一张卡”(Send/Recv)，主要用于流水线并行传激活。

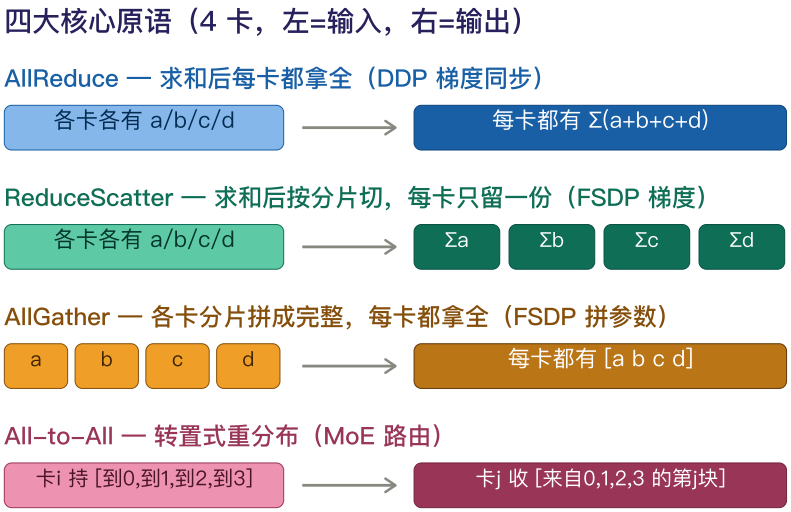
判别口诀：哪种并行策略，决定用哪种通信。DP→AllReduce；FSDP→ReduceScatter+AllGather；TP→AllReduce/AllGather；PP→P2P；MoE→All-to-All。几乎所有通信都能且应与计算 overlap。

地基：通信走哪条物理链路

链路	典型单向带宽	用途
NVLink/NVSwitch(机内)	~450 GB/s (H100)	同机 8 卡，TP 命脉
PCIe(机内)	~64 GB/s (Gen5)	GPU–CPU、offload
IB/RoCE(跨机 RDMA)	~50 GB/s (400Gbps)	跨节点，PP/DP 通信
以太网	更低	一般不用于训练热路径

核心认知：同一次 AllReduce，走 NVLink 和走跨机 IB 慢几个数量级。“哪种并行放机内、哪种跨机”是布局的头等大事。

关键图：四大核心集合通信原语



关键恒等式（务必记住）

AllReduce = ReduceScatter + AllGather（FSDP 拆开 DDP 的数学本质）

Reduce = 求和到一张卡；Gather = 收集到一张卡（原语的“单点”版）

AllReduce —— 数据并行命脉

语义：每张卡有一份等长数据（通常是梯度），逐元素求和，结果每张卡都得到一份完整的和。

用在哪：DDP 梯度同步。N 卡各自在不同 mini-batch 上算梯度，反向后 AllReduce 得全局平均梯度，各自更新出一致参数。

例子：训练 Llama 用 DDP 起 128 卡，每卡吃不同数据，每 step 反向后对全部梯度(7B fp16=14GB)做一次 AllReduce。

通信量：Ring AllReduce 每卡 ≈ 2M，几乎与卡数 N 无关（能 scale 千卡的原因）。

能否 overlap：能，且是头等大事。反向逐层从后往前，某层梯度算完就能立刻 AllReduce，同时继续算前面层。PyTorch DDP 用 gradient bucketing 自动实现。

ReduceScatter + AllGather —— FSDP/ZeRO 两把刀

核心恒等式：AllReduce = ReduceScatter + AllGather，这是理解 FSDP 的钥匙。

ReduceScatter：各卡数据求和后按分片切开，每卡只留 1/N。
AllGather：每卡持一个分片，通信后每卡拿到拼接的完整数据。

用在哪：FSDP/ZeRO-3。前向/反向前用 AllGather 拼回分片参数；反向算完梯度用 ReduceScatter 求和并切片，每卡留 1/N。

为什么不用 AllReduce：AllReduce 后每卡都拿完整梯度仍冗余；ReduceScatter 让每卡只拿 1/N，配合分片优化器状态，显存降到 1/N。FSDP 就是把 DDP 的一次 AllReduce 拆成 ReduceScatter + 平时的 AllGather。

能否 overlap：能，靠 prefetch。算第 i 层时提前 AllGather 第 i+1 层参数；forward_prefetch/backward_prefetch 控制。

All-to-All —— MoE 专属

语义：最"拧巴"的一个。每张卡都持有要发给所有其他卡的不同数据块；通信后每张卡收到来自所有其他卡发给自己的块。本质是跨卡的"矩阵转置式重分布"。

用在哪：MoE 的 Expert Parallel。不同专家在不同卡上，token 经 router 后要发往选中专家所在的卡（dispatch 一次 All-to-All），专家算完再送回（combine 一次 All-to-All）。

例子：训练 Mixtral、DeepSeek-MoE，每个 MoE 层两次 All-to-All。专家跨机时走 IB，极易成瓶颈。

能否 overlap：能但更难。依赖 router 输出（先知道每个 token 去哪），依赖链紧。先进实现(DualPipe/Tutel)会把 dispatch 与其他专家/dense 计算重叠，工程复杂度高于 AllReduce overlap。

P2P Send/Recv —— 流水线并行接力棒

语义：点对点，rank i 把一块数据 Send 给 rank j，rank j Recv。只涉及两张卡。

用在哪：PP 的 stage 间传激活。模型按层切成若干 stage 放不同卡，前向时 stage k 算完把激活 Send 给 stage k+1；反向把梯度 Send 回 stage k。

例子：把 80 层切 8 个 stage，每 stage 10 层，激活沿流水线逐级 Send/Recv，配合 micro-batch 做 1F1B 调度。

通信特点：量相对小(只传激活)，但存在流水线气泡(bubble)——stage 间有依赖，启动收尾阶段部分卡闲置。

能否 overlap：能，这正是流水线调度核心。1F1B 让不同 micro-batch 的前向反向在不同 stage 交错，用一个 micro-batch 的计算掩盖另一个的 P2P 传输，压缩气泡。

overlap 的统一判据

```
能 overlap 的充要条件：
  发起通信 C 后，存在不依赖 C 结果的独立计算 X 可立刻做 → C 与 X 并行
反例（无法 overlap）：
  下一步计算立刻要用这次通信的结果 → 只能干等（暴露在关键路径）

对照：
  DDP AllReduce  → 易（反向逐层有独立计算）
  FSDP AllGather → 易（prefetch 下一层）
  TP AllReduce   → 难（下一步立刻要用，依赖紧 → 必须放 NVLink）
  PP P2P         → 靠 1F1B 调度
  MoE All-to-All → 能但需精心编排

物理下界：单步时间 ≥ max(总计算，总通信)
```

数值算例：感受通信量级

```
DDP 训练 7B 模型，fp16 梯度，64 卡：
  梯度总量 M = 7e9 × 2 Byte = 14 GB
  Ring AllReduce 每卡通信量 ≈ 2M = 28 GB
  跨机 IB 有效带宽 ≈ 40 GB/s：
    T_comm ≈ 28 / 40 ≈ 0.7 s
  若单步计算 T_comp ≈ 1.0 s：
    无 overlap : 1.0 + 0.7 = 1.7 s
    完美 overlap: max(1.0,0.7) = 1.0 s → 加速约 1.7×

结论：通信占计算的比例(70%)决定 overlap 收益上限。
```

五策略通信对照 + 布局原则

DDP : AllReduce | 每step一次(分bucket) | 能overlap(bucketing)

FSDP : AllGather+ReduceScatter| 每层多次 | 能overlap(prefetch)

TP : AllReduce/AllGather | 每层多次(极重) | 难overlap(在关键路径)

PP : P2P Send/Recv | 每micro-batch一次 | 能overlap(1F1B)

MoE(EP) : All-to-All | 每MoE层两次 | 能但复杂

布局原则：越重越频繁的通信放越快的链路。典型 3D 并行：

TP → 机内 8 卡走 NVLink（每层通信最重）

PP → 跨机（P2P 轻，IB 扛得住）

DP/FSDP → 最外层跨机（AllReduce/AllGather 频率较低）

常见误区

1) 误以为 AllReduce 通信量随卡数线性增长：错，Ring 每卡趋近 2M，几乎与 N 无关。
2) 误以为 FSDP 比 DDP 通信更省：反了，FSDP 约 1.5 倍，省的是显存不是通信。
3) 把 TP 放跨机：严重错误，TP 每层通信、依赖紧、难 overlap，必须锁 NVLink 域内(≤8 卡)。
4) 以为开了 overlap 就一定生效：若通信 kernel 与计算抢 SM，或忘用独立 stream/pinned memory，overlap 名存实亡，必须 profiler 验证。
5) All-to-All 当廉价操作：MoE 跨机 All-to-All 常是瓶颈，专家负载不均会拥塞。